PROJECT REPORT

DIGITAL VISITOR FEEDBACK & EXPERIENCE MANAGEMENT PORTAL

TEAM NAME: TECH HACK

ABSTRACT

The Digital Visitor Feedback & Experience Management Portal is a web-based application developed to collect, manage, track, and improve visitor feedback in an organized manner. The system provides separate access for Visitors, Staff members, and Administrators. Visitors can create accounts, log in, and submit feedback. Administrators can manage staff and monitor feedback records, while Staff members can access their assigned responsibilities. The application uses a React frontend, a Node.js and Express.js backend, and a MySQL database. JWT-based authentication and bcryptjs password hashing are used to provide secure access. The system reduces manual work, improves accountability, and supports faster handling of visitor concerns.

1. INTRODUCTION

Visitor feedback is important for understanding user experience and improving the quality of services. Traditional feedback methods such as paper forms and manual registers are difficult to organize, track, and analyze. The proposed portal digitizes the feedback process and provides role-based access to ensure that each user can access only the functions relevant to their role.

The system supports Visitor registration and login, Staff login, Administrator login, secure authentication, feedback submission, database storage, and role-based dashboard access.

2. PROBLEM STATEMENT

Manual feedback collection may lead to delayed responses, lost records, duplicate entries, and difficulty in tracking the status of complaints or suggestions. There is a need for a centralized digital system that securely stores visitor information and feedback, supports role-based access, and enables efficient feedback management.

3. OBJECTIVES

- To develop a secure web-based visitor feedback management system.
- To provide Visitor registration and login functionality.
- To provide separate access for Visitors, Staff, and Administrators.
- To implement role-based authorization and protected dashboards.

- To securely store user information and feedback in MySQL.
- To allow authenticated Visitors to submit feedback.
- To generate unique feedback reference IDs for tracking.
- To support staff management through the Administrator dashboard.
- To improve transparency and efficiency in feedback handling.

4. EXISTING SYSTEM

In a traditional system, feedback is commonly collected using paper forms, registers, emails, or separate spreadsheets. Such methods require manual processing and make it difficult to track feedback status. Data may be misplaced, access control may be weak, and generating reports may take additional time.

5. PROPOSED SYSTEM

The proposed system is a centralized web portal with separate modules for Visitors, Staff, and Administrators. Visitors can create accounts and submit feedback. Staff and Administrator accounts are controlled through authorized administration rather than public registration. The system uses secure login, JWT authentication, password hashing, MySQL storage, and role-based route protection.

New feedback records are stored with a unique reference ID. The initial ticket state follows the existing database design: status is set to Open, assigned staff is set to Unassigned, escalation status is set to Normal, and days pending is initialized to 0.

6. SYSTEM REQUIREMENTS

6.1 Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB or above
- Storage: Minimum 1 GB free space
- Internet connection for development and testing

6.2 Software Requirements

- Operating System: Windows 10/11
- Frontend: React with Vite
- Backend: Node.js and Express.js
- Database: MySQL
- Development Environment: Antigravity IDE or a compatible code editor
- Browser: Google Chrome or Microsoft Edge
- Version Control: Git and GitHub

7. SYSTEM ARCHITECTURE

The application follows a three-tier architecture:

- Presentation Layer: React frontend pages, forms, dashboards, and user interface components.
- Application Layer: Node.js and Express.js APIs for authentication, authorization, validation, feedback processing, and business logic.
- Data Layer: MySQL database for users, feedback tickets, notifications, and system settings.

Overall flow: User → React Frontend → Express API → MySQL Database → API Response → Frontend Dashboard.

8. MODULES

8.1 Authentication Module

The authentication module supports Visitor registration, Visitor login, Staff login, and Administrator login. Passwords are hashed using bcryptjs before being stored. JWT tokens are generated after successful login and are used to access protected APIs. The login response uses the role stored in the database.

8.2 Role-Based Access Control

Protected routes prevent unauthorized access. Visitors are directed to the Visitor dashboard, Staff members to the Staff dashboard, and Administrators to the Admin dashboard. A Visitor cannot access the Admin dashboard by directly entering its URL.

8.3 Administrator and Staff Management

Public registration is restricted to Visitors. Staff accounts are created through authorized Administrator functions. This prevents unauthorized users from creating Staff or Administrator accounts.

8.4 Visitor Feedback Module

Authenticated Visitors can submit feedback using fields such as department, feedback type, subject, description, priority, incident date, and an optional image. The backend associates the feedback with the logged-in Visitor and stores it in MySQL.

8.5 Feedback Tracking

Each feedback ticket has a unique reference ID. The database maintains fields for assigned staff, status, escalation status, days pending, and other details required for future tracking and management.

9. DATABASE DESIGN

9.1 Users Table

The users table stores user ID, user code, name, email, hashed password, role, department, phone number, account status, and timestamps. Roles are Visitor, Staff, and Administrator.

9.2 Feedback Table

The feedback table stores feedback ID, reference ID, visitor information, department, assigned staff, feedback type, subject, description, priority, incident date, status, escalation details, optional image URL, and timestamps.

9.3 Notifications Table

The notifications table stores user-specific notification titles, descriptions, types, read status, and creation time.

9.4 Settings Table

The settings table stores institution information, support details, escalation configuration, notification preferences, and theme settings.

10. TECHNOLOGY STACK

- React: Used to build the interactive frontend.
- Vite: Used for frontend development and build tooling.
- JavaScript: Used for frontend and backend programming.
- Node.js: Used as the backend runtime environment.
- Express.js: Used to develop REST APIs.
- MySQL: Used for persistent data storage.
- mysql2: Used to connect Node.js with MySQL.
- bcryptjs: Used to hash and verify passwords.
- jsonwebtoken: Used to generate and verify JWT tokens.
- Multer: Used for optional image upload handling.
- Git and GitHub: Used for version control and project backup.

11. IMPLEMENTATION

The frontend communicates with the backend through HTTP API requests. Registration data is validated and stored in MySQL. Passwords are hashed before storage. During login, the backend finds the user by email, compares the entered password with the stored hash, checks the account status, and returns a JWT token and user information when authentication succeeds.

The frontend stores the authentication information and uses the token in the Authorization header for protected requests. Role-based routing directs users to the appropriate dashboard.

For feedback submission, the frontend sends the feedback data to the backend. The backend validates the request, obtains the authenticated Visitor identity from the JWT, generates a reference ID, and stores the feedback record in the existing feedback table.

12. TESTING

12.1 Authentication Testing

- Visitor registration was tested with new user details.
- Registered user information was verified in MySQL.
- Visitor login was tested using the registered credentials.
- Administrator login was tested using the seeded Administrator account.
- Staff account creation by an Administrator was tested.
- Staff login was tested using the created Staff account.
- Unauthorized dashboard access was checked using role-based route protection.

12.2 Feedback Testing

- Feedback form validation was checked.
- Authenticated Visitor feedback submission was tested.
- Feedback storage in the MySQL feedback table was verified.
- Unique reference ID generation was checked.
- Default values such as Open status and Unassigned staff were verified.

13. RESULTS

The authentication system was successfully integrated with the React frontend, Express backend, and MySQL database. Visitor registration and login worked correctly, and user data was stored in the database. Administrator and Staff access was separated using role-based authorization. Administrator-controlled Staff account creation was implemented. The feedback module was designed to securely associate submitted feedback with the logged-in Visitor and store ticket information using the existing database schema.

14. ADVANTAGES

- Centralized storage of user and feedback information.
- Secure authentication using JWT and bcryptjs.
- Role-based access control.
- Reduced manual work and improved record management.
- Unique feedback reference IDs for tracking.
- Better accountability through staff assignment and status management.
- Scalable architecture for future modules.

15. LIMITATIONS

The current implementation may require additional work for complete notification delivery, advanced analytics, automated escalation, email integration, and deployment. These features can be added in future versions.

16. FUTURE ENHANCEMENTS

- Email and in-app notifications.
- Automatic escalation based on pending days.
- Advanced analytics and graphical reports.
- Feedback search, filtering, and export.
- Real-time status updates.
- Mobile-responsive improvements and a dedicated mobile application.
- Admin-configurable escalation rules.
- Cloud deployment and production monitoring.

17. CONCLUSION

The Digital Visitor Feedback & Experience Management Portal provides a secure and organized approach to visitor feedback management. The use of React, Node.js, Express.js, and MySQL enables a clear separation between the user interface, application logic, and data storage. JWT authentication, bcryptjs password hashing, and role-based authorization improve security. The system supports Visitor registration, controlled Staff and Administrator access, and structured feedback handling. The project can be extended with notifications, analytics, escalation, and reporting features to provide a complete feedback management solution.

18. REFERENCES

- React Documentation.
- Node.js Documentation.
- Express.js Documentation.
- MySQL Documentation.
- JSON Web Token Documentation.
- bcryptjs Documentation.